Cadeias de Markov II

Avaliação de Desempenho (PC010029) · Aula 09

Prof. Dr. Helvecio Bezerra Leal Neto

Universidade Federal do Oeste do Pará

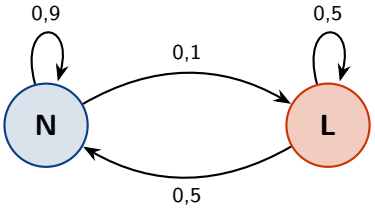
27 de outubro de 2026

Revisão: cadeia de Markov

Definições

- **Estado:** a situação do sistema em um instante, escolhida em um conjunto finito de possibilidades.
- **Passo:** o intervalo entre duas observações consecutivas.
- **Cadeia de Markov:** sequência de estados em que a probabilidade do próximo estado depende apenas do estado atual, não do caminho percorrido.
- **Matriz de transição P:** p_{ij} é a probabilidade de passar do estado i ao estado j em um passo. Cada linha soma 1.

Revisão: o servidor web da Aula 08



Estados: resposta **N**ormal ou **L**enta.
Passo: um minuto.

$$P = \begin{bmatrix} 0{,}9 & 0{,}1 \\ 0{,}5 & 0{,}5 \end{bmatrix}$$

Linha N: de um minuto normal, o seguinte é normal com probabilidade 0,9 e lento com 0,1. Linha L: de um minuto lento, volta ao normal com 0,5 e continua lento com 0,5.

Distribuição estacionária π

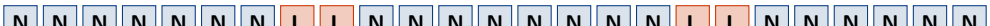
Fração do tempo que a cadeia passa em cada estado a longo prazo; satisfaz $\pi P = \pi$. Para o servidor, $\pi = [0{,}83 \quad 0{,}17]$: lento em 17% do tempo, cerca de 10 minutos por hora.

O registro de estados

Definições

- **Monitor por amostragem** (Aula 06): observa o sistema em instantes fixos e anota o estado encontrado.
- **Registro** (*log*): a sequência de estados anotados, $x_1, x_2, \ldots, x_T$, um por passo.

Para o servidor web, um registro de 25 minutos, com um estado por minuto:



minuto 1 minuto 25

Questão: a partir apenas deste registro, como obter a matriz de transição P?

O estimador da matriz de transição

Definição

Seja n_{ij} o número de pares consecutivos $(x_t = i,\ x_{t+1} = j)$ do registro, e $n_i = \sum_j n_{ij}$ o número de pares que começam em i. A estimativa de p_{ij} é

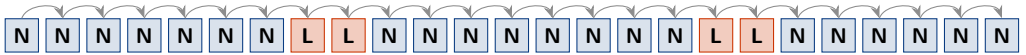
$$\hat{p}_{ij} = \frac{n_{ij}}{n_i}.$$

Em palavras: entre as vezes em que o sistema estava no estado i, a fração em que o estado seguinte foi j.

Procedimento:

1. ler o registro em pares consecutivos (atual, seguinte);
2. contar as ocorrências de cada tipo de par, n_{ij};
3. dividir cada contagem pelo total da sua linha, n_i.

Exemplo: contagem das transições

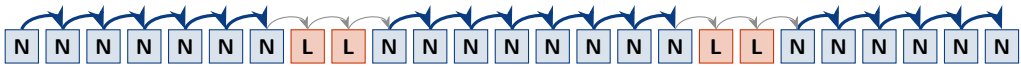


1. Cada seta liga um minuto ao seguinte: um **par** (atual, seguinte). 25 minutos geram **24 pares**.

de \ para	N	L
N		
L		

tabela de contagens n_{ij}

Exemplo: contagem das transições

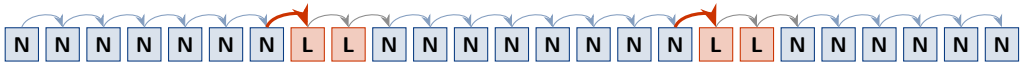


1. Cada seta liga um minuto ao seguinte: um **par** (atual, seguinte). 25 minutos geram **24 pares**.
2. Pares **N→N**: 18.

de \ para	N	L
N	**18**	
L		

tabela de contagens n_{ij}

Exemplo: contagem das transições

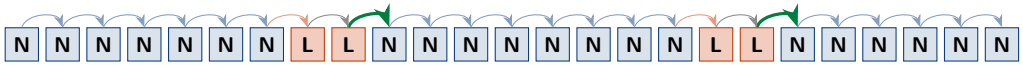


1. Cada seta liga um minuto ao seguinte: um **par** (atual, seguinte). 25 minutos geram **24 pares**.
2. Pares **N→N**: 18.
3. Pares **N→L**: 2 (minutos 7 e 17).

de \ para	N	L
N	**18**	**2**
L		

tabela de contagens n_{ij}

Exemplo: contagem das transições

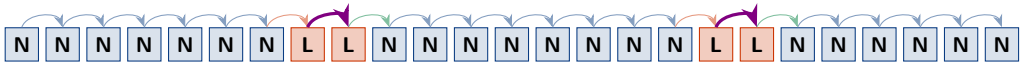


1. Cada seta liga um minuto ao seguinte: um **par** (atual, seguinte). 25 minutos geram **24 pares**.
2. Pares **N→N**: 18.
3. Pares **N→L**: 2 (minutos 7 e 17).
4. Pares **L→N**: 2 (minutos 9 e 19).

de \ para	N	L
N	**18**	**2**
L	**2**	

tabela de contagens n_{ij}

Exemplo: contagem das transições

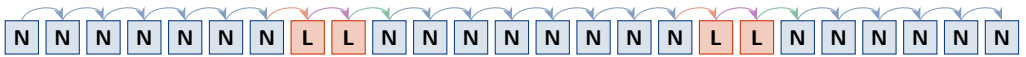


1. Cada seta liga um minuto ao seguinte: um **par** (atual, seguinte). 25 minutos geram **24 pares**.
2. Pares **N→N**: 18.
3. Pares **N→L**: 2 (minutos 7 e 17).
4. Pares **L→N**: 2 (minutos 9 e 19).
5. Pares **L→L**: 2 (minutos 8 e 18).

de \ para	N	L
N	**18**	**2**
L	**2**	**2**

tabela de contagens n_{ij}

Exemplo: contagem das transições



1. Cada seta liga um minuto ao seguinte: um **par** (atual, seguinte). 25 minutos geram **24 pares**.
2. Pares **N→N**: 18.
3. Pares **N→L**: 2 (minutos 7 e 17).
4. Pares **L→N**: 2 (minutos 9 e 19).
5. Pares **L→L**: 2 (minutos 8 e 18).
6. Conferência: $18 + 2 + 2 + 2 = 24$ pares.

de \ para	N	L
N	**18**	**2**
L	**2**	**2**

tabela de contagens n_{ij}

Exemplo: da contagem à matriz estimada

Cada contagem é dividida pelo total da sua linha, n_i, que é o número de vezes em que o sistema estava no estado i:

	contagens n_{ij}		
de \ para	N	L	total n_i
N	18	2	$18 + 2 = \mathbf{20}$
L	2	2	$2 + 2 = \mathbf{4}$

Exemplo: da contagem à matriz estimada

Cada contagem é dividida pelo total da sua linha, n_i, que é o número de vezes em que o sistema estava no estado i:

	contagens n_{ij}				estimativas $\hat{p}_{ij} = n_{ij}/n_i$	
de \ para	N	L	total n_i		N	L
N	18	2	$18 + 2 = \mathbf{20}$	$\Rightarrow$	$18/20 = 0{,}9$	$2/20 = 0{,}1$
L	2	2	$2 + 2 = \mathbf{4}$			

Exemplo: da contagem à matriz estimada

Cada contagem é dividida pelo total da sua linha, n_i, que é o número de vezes em que o sistema estava no estado i:

	contagens n_{ij}				estimativas $\hat{p}_{ij} = n_{ij}/n_i$	
de \ para	N	L	total n_i		N	L
N	18	2	$18 + 2 = \mathbf{20}$	$\Rightarrow$	$18/20 = 0{,}9$	$2/20 = 0{,}1$
L	2	2	$2 + 2 = \mathbf{4}$	$\Rightarrow$	$2/4 = 0{,}5$	$2/4 = 0{,}5$

Exemplo: da contagem à matriz estimada

Cada contagem é dividida pelo total da sua linha, n_i, que é o número de vezes em que o sistema estava no estado i:

	contagens n_{ij}				estimativas $\hat{p}_{ij} = n_{ij}/n_i$	
de \ para	N	L	total n_i		N	L
N	18	2	$18 + 2 = \mathbf{20}$	$\Rightarrow$	$18/20 = 0{,}9$	$2/20 = 0{,}1$
L	2	2	$2 + 2 = \mathbf{4}$	$\Rightarrow$	$2/4 = 0{,}5$	$2/4 = 0{,}5$

$$\hat{P} = \begin{bmatrix} 0{,}9 & 0{,}1 \\ 0{,}5 & 0{,}5 \end{bmatrix}$$

Cada linha soma 1, como em toda matriz de transição. Neste registro, a estimativa coincide com a matriz da Aula 08; o próximo slide mostra que isso não ocorre em geral.

Estimação de P em Python

```
import numpy as np, pandas as pd
rng = np.random.default_rng(42)
P = np.array([[0.9, 0.1], [0.5, 0.5]])
log = [0]                        # 0 = Normal, 1 = Lento
for minuto in range(7 * 24 * 60 - 1):  # uma semana
    log.append(rng.choice(2, p=P[log[-1]]))
# daqui para baixo serve para QUALQUER log real
s = pd.Series(log).map({0: 'N', 1: 'L'})
P_est = pd.crosstab(s[:-1].values, s[1:].values,
          rownames=['de'], colnames=['para'],
          normalize='index')
print(P_est.loc[['N', 'L'], ['N', 'L']].round(3))
```

Estimação de P em Python

```
import numpy as np, pandas as pd
rng = np.random.default_rng(42)
P = np.array([[0.9, 0.1], [0.5, 0.5]])
log = [0]                          # 0 = Normal, 1 = Lento
for minuto in range(7 * 24 * 60 - 1):  # uma semana
    log.append(rng.choice(2, p=P[log[-1]]))
# daqui para baixo serve para QUALQUER log real
s = pd.Series(log).map({0: 'N', 1: 'L'})
P_est = pd.crosstab(s[:-1].values, s[1:].values,
          rownames=['de'], colnames=['para'],
          normalize='index')
print(P_est.loc[['N', 'L'], ['N', 'L']].round(3))
```

Saída:

```
para       N      L
de
N      0.902  0.098
L      0.498  0.502
```

Estimação de P em Python

```
import numpy as np, pandas as pd
rng = np.random.default_rng(42)
P = np.array([[0.9, 0.1], [0.5, 0.5]])
log = [0]                          # 0 = Normal, 1 = Lento
for minuto in range(7 * 24 * 60 - 1):  # uma semana
    log.append(rng.choice(2, p=P[log[-1]]))
# daqui para baixo serve para QUALQUER log real
s = pd.Series(log).map({0: 'N', 1: 'L'})
P_est = pd.crosstab(s[:-1].values, s[1:].values,
          rownames=['de'], colnames=['para'],
          normalize='index')
print(P_est.loc[['N', 'L'], ['N', 'L']].round(3))
```

Saída:

```
para       N      L
de
N      0.902  0.098
L      0.498  0.502
```

Registro simulado, como na Aula 05; com um registro real, troca-se só a primeira parte. `crosstab` conta os pares n_{ij}; `normalize='index'` divide por n_i.

Tempo de permanência em um estado

Definição e resultado

Tempo de permanência: número de passos consecutivos no mesmo estado. Se a probabilidade de sair é p por passo, a permanência média é de $1/p$ passos.

No modelo do servidor web:

- Lento: $p = 0{,}5$, média **2** min.
- Normal: $p = 0{,}1$, média **10** min.

Aplicação ao seminário

Conte os pares (fila longa, fila curta) da sua hora; $1/p$ estima a duração da fila longa.

Tempo de permanência em um estado

Definição e resultado

Tempo de permanência: número de passos consecutivos no mesmo estado. Se a probabilidade de sair é p por passo, a permanência média é de $1/p$ passos.

No modelo do servidor web:

- Lento: $p = 0{,}5$, média **2** min.
- Normal: $p = 0{,}1$, média **10** min.

No registro de uma semana:

episódios de L consecutivos	824
duração média	**2,01 min**
maior episódio	10 min

A média confere; o episódio de 10 min é o que o usuário percebe.

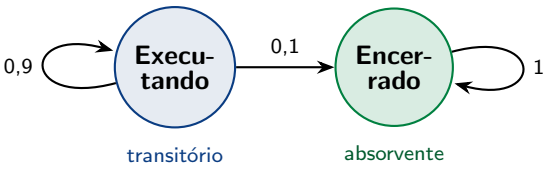
Aplicação ao seminário

Conte os pares (fila longa, fila curta) da sua hora; $1/p$ estima a duração da fila longa.

Definição: estado absorvente

Definições

- **Estado absorvente:** a cadeia entra e **nunca mais sai**. Na matriz, $p_{ii} = 1$.
- **Estado transitório:** estado do qual a cadeia acaba saindo, para não voltar mais.



Exemplo: um processo termina com probabilidade 0,1 a cada passo e, encerrado, não volta a executar.

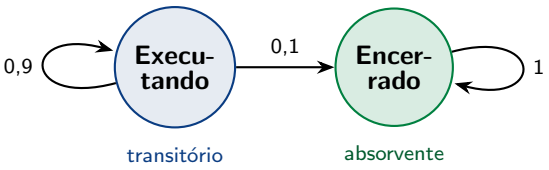
$$P = \begin{bmatrix} 0{,}9 & 0{,}1 \\ 0 & \mathbf{1} \end{bmatrix} \begin{matrix} \leftarrow \text{Executando} \\ \leftarrow \text{Encerrado} \end{matrix}$$

Contraste: no servidor web, de L volta-se a N ($p_{LL} = 0{,}5 < 1$): nenhum estado é absorvente.

Definição: estado absorvente

Definições

- **Estado absorvente:** a cadeia entra e **nunca mais sai**. Na matriz, $p_{ii} = 1$.
- **Estado transitório:** estado do qual a cadeia acaba saindo, para não voltar mais.



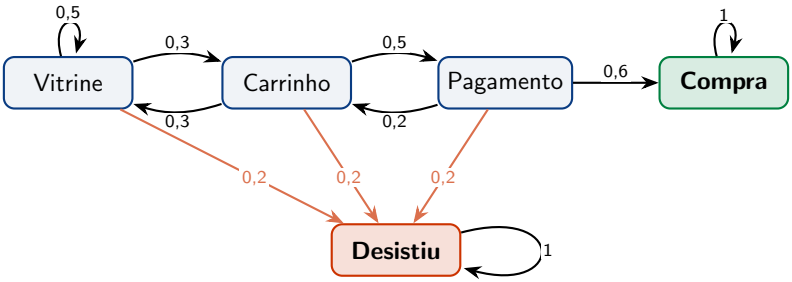
Exemplo: um processo termina com probabilidade 0,1 a cada passo e, encerrado, não volta a executar.

$$P = \begin{bmatrix} 0{,}9 & 0{,}1 \\ 0 & \mathbf{1} \end{bmatrix} \begin{matrix} \leftarrow \text{Executando} \\ \leftarrow \text{Encerrado} \end{matrix}$$

Contraste: no servidor web, de L volta-se a N ($p_{LL} = 0{,}5 < 1$): nenhum estado é absorvente.

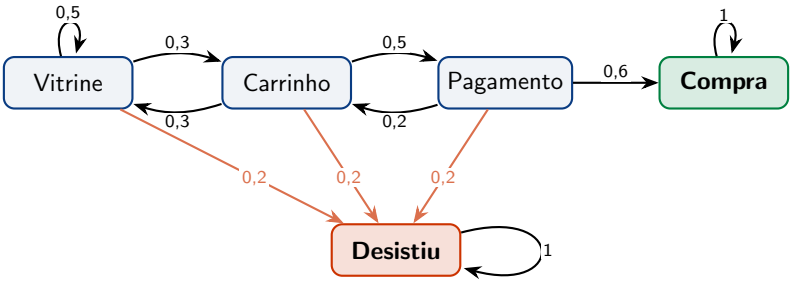
Novas perguntas: em *qual* absorvente a cadeia termina, com que probabilidade e depois de quantos passos?

Um sistema em que o cliente termina



- **Absorventes:** **Compra** e **Desistiu**, com laço de probabilidade 1.
- **Transitórios:** Vitrine, Carrinho e Pagamento. Cada clique é um passo.

Um sistema em que o cliente termina



- **Absorventes:** **Compra** e **Desistiu**, com laço de probabilidade 1.
- **Transitórios:** Vitrine, Carrinho e Pagamento. Cada clique é um passo.

Pergunta: de cada 100 clientes que entram pela Vitrine, quantos compram?

Seguindo 100 clientes, clique a clique

100 clientes entram pela Vitrine. A cada clique, cada grupo se divide conforme as setas do diagrama.

clique	Vitrine	Carrinho	Pagam.	**Compra**	**Desistiu**
0	100	0	0	0	0

Clique 0. Todos os 100 estão na Vitrine.

Seguindo 100 clientes, clique a clique

100 clientes entram pela Vitrine. A cada clique, cada grupo se divide conforme as setas do diagrama.

clique	Vitrine	Carrinho	Pagam.	**Compra**	**Desistiu**
0	100	0	0	0	0
1	50	30	0	0	20

Clique 1. Dos 100 na Vitrine:

- 50 ficam (0,5);
- 30 vão ao Carrinho (0,3);
- 20 desistem (0,2).

Seguindo 100 clientes, clique a clique

100 clientes entram pela Vitrine. A cada clique, cada grupo se divide conforme as setas do diagrama.

clique	Vitrine	Carrinho	Pagam.	Compra	Desistiu
0	100	0	0	0	0
1	50	30	0	0	20
2	34	15	15	0	36

Clique 2. Cada grupo se divide:

- 50 na Vitrine: 25 ficam, 15 ao Carrinho, 10 desistem;
- 30 no Carrinho: 9 voltam à Vitrine, 15 ao Pagamento, 6 desistem.

Vitrine: $25 + 9 = 34$. Desistiu: $20 + 10 + 6 = 36$.

Seguindo 100 clientes, clique a clique

100 clientes entram pela Vitrine. A cada clique, cada grupo se divide conforme as setas do diagrama.

clique	Vitrine	Carrinho	Pagam.	**Compra**	**Desistiu**
0	100	0	0	0	0
1	50	30	0	0	20
2	34	15	15	0	36
3	21,5	13,2	7,5	9	48,8

Clique 3. Dos 15 no Pagamento, 60% compram: as primeiras **9 compras**. Os outros grupos se dividem do mesmo jeito. (Números com vírgula são médias.)

Seguindo 100 clientes, clique a clique

100 clientes entram pela Vitrine. A cada clique, cada grupo se divide conforme as setas do diagrama.

clique	Vitrine	Carrinho	Pagam.	**Compra**	**Desistiu**
0	100	0	0	0	0
1	50	30	0	0	20
2	34	15	15	0	36
3	21,5	13,2	7,5	9	48,8
5	9,7	5,7	4,0	17,5	63,1
10	1,3	0,8	0,6	24,0	73,4
20	0	0	0	**25**	**75**

Com o tempo, todos caem em Compra ou em Desistiu e lá ficam, porque esses estados são absorventes. Depois de uns 20 cliques, o total para de mudar.

Seguindo 100 clientes, clique a clique

100 clientes entram pela Vitrine. A cada clique, cada grupo se divide conforme as setas do diagrama.

clique	Vitrine	Carrinho	Pagam.	**Compra**	**Desistiu**
0	100	0	0	0	0
1	50	30	0	0	20
2	34	15	15	0	36
3	21,5	13,2	7,5	9	48,8
5	9,7	5,7	4,0	17,5	63,1
10	1,3	0,8	0,6	24,0	73,4
20	0	0	0	**25**	**75**

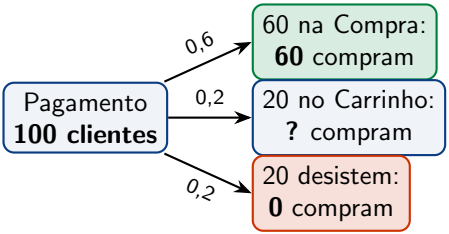
Com o tempo, todos caem em Compra ou em Desistiu e lá ficam, porque esses estados são absorventes. Depois de uns 20 cliques, o total para de mudar.

Resposta: de cada 100 clientes que entram pela Vitrine, **25 compram** e 75 desistem.

Um atalho: a chance de comprar a partir de cada página

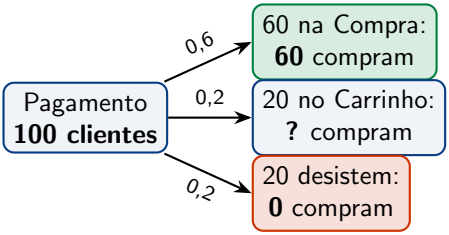
Simular leva muitos cliques. Atalho: x_V, x_C, x_{Pg} = **fração dos clientes que, partindo daquela página, um dia compram**. Exemplo no Pagamento:

O ramo difícil: os 20 no Carrinho.



Um atalho: a chance de comprar a partir de cada página

Simular leva muitos cliques. Atalho: x_V, x_C, x_{Pg} = **fração dos clientes que, partindo daquela página, um dia compram**. Exemplo no Pagamento:

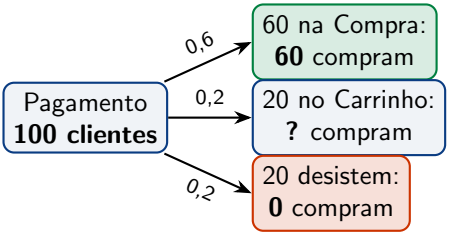


O ramo difícil: os 20 no Carrinho.

- Quantos deles compram? Ainda não sabemos.

Um atalho: a chance de comprar a partir de cada página

Simular leva muitos cliques. Atalho: x_V, x_C, x_{Pg} = **fração dos clientes que, partindo daquela página, um dia compram**. Exemplo no Pagamento:

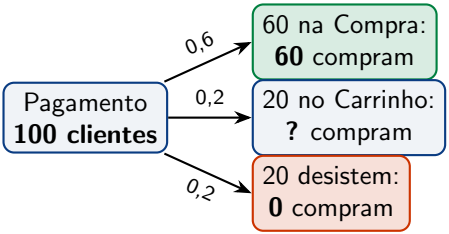


O ramo difícil: os 20 no Carrinho.

- Quantos deles compram? Ainda não sabemos.
- A cadeia **não lembra** de onde vieram (Markov): compram na mesma fração que **qualquer** cliente no Carrinho, ou seja, x_C.

Um atalho: a chance de comprar a partir de cada página

Simular leva muitos cliques. Atalho: x_V, x_C, x_{Pg} = **fração dos clientes que, partindo daquela página, um dia compram**. Exemplo no Pagamento:

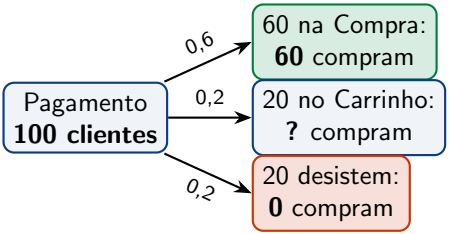


O ramo difícil: os 20 no Carrinho.

- Quantos deles compram? Ainda não sabemos.
- A cadeia **não lembra** de onde vieram (Markov): compram na mesma fração que **qualquer** cliente no Carrinho, ou seja, x_C.
- Se x_C fosse 0,4, seriam $20 \times 0{,}4 = 8$. Em geral, $\mathbf{20 \times x_C}$.

Um atalho: a chance de comprar a partir de cada página

Simular leva muitos cliques. Atalho: x_V, x_C, x_{Pg} = **fração dos clientes que, partindo daquela página, um dia compram**. Exemplo no Pagamento:



O ramo difícil: os 20 no Carrinho.

- Quantos deles compram? Ainda não sabemos.
- A cadeia **não lembra** de onde vieram (Markov): compram na mesma fração que **qualquer** cliente no Carrinho, ou seja, x_C.
- Se x_C fosse 0,4, seriam $20 \times 0{,}4 = 8$. Em geral, $\mathbf{20} \times \mathbf{x_C}$.

Compram $60 + 20\,x_C$ de 100, logo $\boxed{x_{Pg} = 0{,}6 + 0{,}2\,x_C}$

As três equações e a solução

$$x_{Pg} = 0{,}6 + 0{,}2\,x_C \qquad x_V = 0{,}5\,x_V + 0{,}3\,x_C \qquad x_C = 0{,}3\,x_V + 0{,}5\,x_{Pg}$$

x_V aparece dos dois lados porque metade de quem está na Vitrine continua nela.

Resolução por substituição:

As três equações e a solução

$$x_{Pg} = 0{,}6 + 0{,}2\,x_C \qquad x_V = 0{,}5\,x_V + 0{,}3\,x_C \qquad x_C = 0{,}3\,x_V + 0{,}5\,x_{Pg}$$

x_V aparece dos dois lados porque metade de quem está na Vitrine continua nela.

Resolução por substituição:

1. **Vitrine:** $x_V - 0{,}5\,x_V = 0{,}3\,x_C$, logo $0{,}5\,x_V = 0{,}3\,x_C$ e $x_V = 0{,}6\,x_C$.

As três equações e a solução

$$x_{Pg} = 0{,}6 + 0{,}2\,x_C \qquad x_V = 0{,}5\,x_V + 0{,}3\,x_C \qquad x_C = 0{,}3\,x_V + 0{,}5\,x_{Pg}$$

x_V aparece dos dois lados porque metade de quem está na Vitrine continua nela.

Resolução por substituição:

1. **Vitrine:** $x_V - 0{,}5\,x_V = 0{,}3\,x_C$, logo $0{,}5\,x_V = 0{,}3\,x_C$ e $x_V = 0{,}6\,x_C$.
2. **Carrinho**, trocando x_V e x_{Pg}:
   $x_C = 0{,}3\,(0{,}6\,x_C) + 0{,}5\,(0{,}6 + 0{,}2\,x_C) = 0{,}28\,x_C + 0{,}3$.
   Então $0{,}72\,x_C = 0{,}3$ e $x_C = 0{,}3/0{,}72 \approx 0{,}417$.

As três equações e a solução

$$x_{Pg} = 0{,}6 + 0{,}2\,x_C \qquad x_V = 0{,}5\,x_V + 0{,}3\,x_C \qquad x_C = 0{,}3\,x_V + 0{,}5\,x_{Pg}$$

x_V aparece dos dois lados porque metade de quem está na Vitrine continua nela.

Resolução por substituição:

1. **Vitrine:** $x_V - 0{,}5\,x_V = 0{,}3\,x_C$, logo $0{,}5\,x_V = 0{,}3\,x_C$ e $x_V = 0{,}6\,x_C$.
2. **Carrinho**, trocando x_V e x_{Pg}:
   $x_C = 0{,}3\,(0{,}6\,x_C) + 0{,}5\,(0{,}6 + 0{,}2\,x_C) = 0{,}28\,x_C + 0{,}3$.
   Então $0{,}72\,x_C = 0{,}3$ e $x_C = 0{,}3/0{,}72 \approx 0{,}417$.
3. **Voltando:** $x_V = 0{,}6 \times 0{,}417 = \mathbf{0{,}25}$ e $x_{Pg} = 0{,}6 + 0{,}2 \times 0{,}417 \approx 0{,}683$.

As três equações e a solução

$$x_{Pg} = 0{,}6 + 0{,}2\,x_C \qquad x_V = 0{,}5\,x_V + 0{,}3\,x_C \qquad x_C = 0{,}3\,x_V + 0{,}5\,x_{Pg}$$

x_V aparece dos dois lados porque metade de quem está na Vitrine continua nela.

Resolução por substituição:

1. **Vitrine:** $x_V - 0{,}5\,x_V = 0{,}3\,x_C$, logo $0{,}5\,x_V = 0{,}3\,x_C$ e $x_V = 0{,}6\,x_C$.
2. **Carrinho**, trocando x_V e x_{Pg}:
   $x_C = 0{,}3\,(0{,}6\,x_C) + 0{,}5\,(0{,}6 + 0{,}2\,x_C) = 0{,}28\,x_C + 0{,}3$.
   Então $0{,}72\,x_C = 0{,}3$ e $x_C = 0{,}3/0{,}72 \approx 0{,}417$.
3. **Voltando:** $x_V = 0{,}6 \times 0{,}417 = \mathbf{0{,}25}$ e $x_{Pg} = 0{,}6 + 0{,}2 \times 0{,}417 \approx 0{,}683$.

Resultado: $x_V = 0{,}25$, os mesmos **25 compradores em 100** da simulação, agora sem simular. Quem chega ao Pagamento compra em 68% dos casos.

A biblioteca PyDTMC

O que é

PyDTMC (*Python Discrete-Time Markov Chains*): biblioteca Python gratuita e de código aberto para **cadeias de Markov de tempo discreto**. Recebe P e os nomes dos estados e faz os cálculos. Instala-se com `pip install PyDTMC`.

O que se quer	**Na PyDTMC**	**Visto em**
criar a cadeia	`mc = MarkovChain(P, estados)`	
distribuição estacionária π	`mc.pi`	Aula 08
quais estados são absorventes	`mc.absorbing_states`	este bloco
probabilidade de absorção	`mc.absorption_probabilities()`	este bloco
passos até a absorção	`mc.absorption_times()`	próximo slide
simular uma trajetória	`mc.simulate(n)`	Aula 05

A loja virtual na PyDTMC

```
from pydtmc import MarkovChain
estados = ['Vitrine', 'Carrinho', 'Pagamento',
           'Compra', 'Desistiu']
p = [[0.5, 0.3, 0.0, 0.0, 0.2],   # Vitrine
     [0.3, 0.0, 0.5, 0.0, 0.2],   # Carrinho
     [0.0, 0.2, 0.0, 0.6, 0.2],   # Pagamento
     [0.0, 0.0, 0.0, 1.0, 0.0],   # Compra
     [0.0, 0.0, 0.0, 0.0, 1.0]]   # Desistiu
mc = MarkovChain(p, estados)
print(mc.absorbing_states)
print(mc.absorption_probabilities().round(3))
print(mc.absorption_times().round(2))
```

A loja virtual na PyDTMC

```
from pydtmc import MarkovChain
estados = ['Vitrine', 'Carrinho', 'Pagamento',
           'Compra', 'Desistiu']
p = [[0.5, 0.3, 0.0, 0.0, 0.2],   # Vitrine
     [0.3, 0.0, 0.5, 0.0, 0.2],   # Carrinho
     [0.0, 0.2, 0.0, 0.6, 0.2],   # Pagamento
     [0.0, 0.0, 0.0, 1.0, 0.0],   # Compra
     [0.0, 0.0, 0.0, 0.0, 1.0]]   # Desistiu
mc = MarkovChain(p, estados)
print(mc.absorbing_states)
print(mc.absorption_probabilities().round(3))
print(mc.absorption_times().round(2))
```

Saída:

```
['Compra', 'Desistiu']
[[0.25  0.417 0.683]
 [0.75  0.583 0.317]]
[3.75 2.92 1.58]
```

A loja virtual na PyDTMC

```
from pydtmc import MarkovChain
estados = ['Vitrine', 'Carrinho', 'Pagamento',
           'Compra', 'Desistiu']
p = [[0.5, 0.3, 0.0, 0.0, 0.2],   # Vitrine
     [0.3, 0.0, 0.5, 0.0, 0.2],   # Carrinho
     [0.0, 0.2, 0.0, 0.6, 0.2],   # Pagamento
     [0.0, 0.0, 0.0, 1.0, 0.0],   # Compra
     [0.0, 0.0, 0.0, 0.0, 1.0]]   # Desistiu
mc = MarkovChain(p, estados)
print(mc.absorbing_states)
print(mc.absorption_probabilities().round(3))
print(mc.absorption_times().round(2))
```

Saída:

```
['Compra', 'Desistiu']
[[0.25  0.417 0.683]
 [0.75  0.583 0.317]]
[3.75 2.92 1.58]
```

- Matriz, linha 1: chance de **Compra** saindo de Vitrine, Carrinho e Pagamento; os **25%** conferem.

A loja virtual na PyDTMC

```
from pydtmc import MarkovChain
estados = ['Vitrine', 'Carrinho', 'Pagamento',
           'Compra', 'Desistiu']
p = [[0.5, 0.3, 0.0, 0.0, 0.2],   # Vitrine
     [0.3, 0.0, 0.5, 0.0, 0.2],   # Carrinho
     [0.0, 0.2, 0.0, 0.6, 0.2],   # Pagamento
     [0.0, 0.0, 0.0, 1.0, 0.0],   # Compra
     [0.0, 0.0, 0.0, 0.0, 1.0]]   # Desistiu
mc = MarkovChain(p, estados)
print(mc.absorbing_states)
print(mc.absorption_probabilities().round(3))
print(mc.absorption_times().round(2))
```

Saída:

```
['Compra', 'Desistiu']
[[0.25  0.417 0.683]
 [0.75  0.583 0.317]]
[3.75 2.92 1.58]
```

- Matriz, linha 1: chance de **Compra** saindo de Vitrine, Carrinho e Pagamento; os **25%** conferem.
- Linha 2: chance de **Desistiu**.

A loja virtual na PyDTMC

```
from pydtmc import MarkovChain
estados = ['Vitrine', 'Carrinho', 'Pagamento',
           'Compra', 'Desistiu']
p = [[0.5, 0.3, 0.0, 0.0, 0.2],   # Vitrine
     [0.3, 0.0, 0.5, 0.0, 0.2],   # Carrinho
     [0.0, 0.2, 0.0, 0.6, 0.2],   # Pagamento
     [0.0, 0.0, 0.0, 1.0, 0.0],   # Compra
     [0.0, 0.0, 0.0, 0.0, 1.0]]   # Desistiu
mc = MarkovChain(p, estados)
print(mc.absorbing_states)
print(mc.absorption_probabilities().round(3))
print(mc.absorption_times().round(2))
```

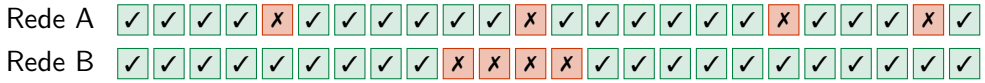
Saída:

```
['Compra', 'Desistiu']
[[0.25  0.417 0.683]
 [0.75  0.583 0.317]]
[3.75 2.92 1.58]
```

- Matriz, linha 1: chance de **Compra** saindo de Vitrine, Carrinho e Pagamento; os **25%** conferem.
- Linha 2: chance de **Desistiu**.
- Último vetor: cliques até terminar, em média; **3,75** da Vitrine.

Perdas em rajada: o modelo de Gilbert

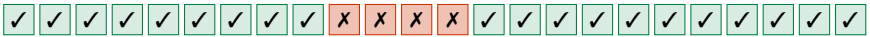
Dois enlaces perdem **4 de cada 24 pacotes** (exagerado, para ver melhor):



Perdas em rajada: o modelo de Gilbert

Dois enlaces perdem **4 de cada 24 pacotes** (exagerado, para ver melhor):

Rede A 

Rede B 

Mesma perda média, efeito diferente: uma chamada de vídeo disfarça perdas isoladas (A), mas **congela** na rajada (B).

Perdas em rajada: o modelo de Gilbert

Dois enlaces perdem **4 de cada 24 pacotes** (exagerado, para ver melhor):

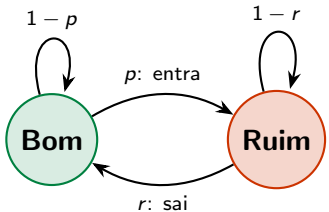
Rede A ✓ ✓ ✓ ✓ ✗ ✓ ✓ ✓ ✓ ✓ ✓ ✗ ✓ ✓ ✓ ✓ ✓ ✓ ✗ ✓ ✓ ✓ ✗ ✓

Rede B ✓ ✓ ✓ ✓ ✓ ✓ ✓ ✓ ✓ ✗ ✗ ✗ ✗ ✓ ✓ ✓ ✓ ✓ ✓ ✓ ✓ ✓ ✓ ✓

Mesma perda média, efeito diferente: uma chamada de vídeo disfarça perdas isoladas (A), mas **congela** na rajada (B).

Definição: modelo de Gilbert

Cadeia de Markov de **dois estados** proposta por E. N. Gilbert (Bell Labs, 1960) para canais com erros em rajada. O estado é a condição do canal: **Bom**, o pacote chega; **Ruim** (interferência, sinal fraco), o pacote se perde. **Passo:** um pacote.



Mesma estrutura do servidor web (N/L).

Modelo de Gilbert: duas medidas

1. Quanto se perde? A longo prazo, **entram** no Ruim tantos quanto **saem**:

$$\underbrace{\pi_{\mathrm{B}}\, p}_{\text{entram}} = \underbrace{\pi_{\mathrm{R}}\, r}_{\text{saem}} \;\Rightarrow\; \pi_{\mathrm{R}} = \frac{p}{p+r}$$

Modelo de Gilbert: duas medidas

1. Quanto se perde? A longo prazo, **entram** no Ruim tantos quanto **saem**:

$$\underbrace{\pi_{\mathrm{B}}\, p}_{\text{entram}} = \underbrace{\pi_{\mathrm{R}}\, r}_{\text{saem}} \;\Rightarrow\; \pi_{\mathrm{R}} = \frac{p}{p+r}$$

2. Quanto dura uma rajada? Mesma regra do tempo de permanência: com chance r de sair a cada pacote, a fase Ruim dura em média $1/r$ **pacotes**.

Modelo de Gilbert: duas medidas

1. Quanto se perde? A longo prazo, **entram** no Ruim tantos quanto **saem**:

$$\underbrace{\pi_B\, p}_{\text{entram}} = \underbrace{\pi_R\, r}_{\text{saem}} \;\Rightarrow\; \pi_R = \frac{p}{p+r}$$

2. Quanto dura uma rajada? Mesma regra do tempo de permanência: com chance r de sair a cada pacote, a fase Ruim dura em média $1/r$ **pacotes**.

	p	r	**perda média**, $\frac{p}{p+r}$	**rajada média**, $\frac{1}{r}$
Rede A	0,0101	0,99	1%	1 pacote
Rede B	0,002	0,2	1%	**5 pacotes**

Modelo de Gilbert: duas medidas

1. Quanto se perde? A longo prazo, **entram** no Ruim tantos quanto **saem**:

$$\underbrace{\pi_B\, p}_{\text{entram}} = \underbrace{\pi_R\, r}_{\text{saem}} \;\Rightarrow\; \pi_R = \frac{p}{p+r}$$

2. Quanto dura uma rajada? Mesma regra do tempo de permanência: com chance r de sair a cada pacote, a fase Ruim dura em média $1/r$ **pacotes**.

	p	r	**perda média**, $\frac{p}{p+r}$	**rajada média**, $\frac{1}{r}$
Rede A	0,0101	0,99	1%	1 pacote
Rede B	0,002	0,2	1%	**5 pacotes**

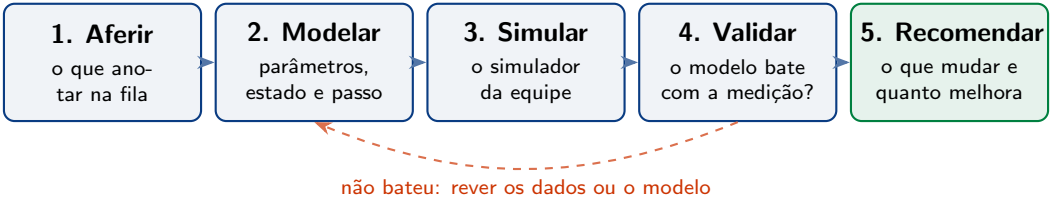
Conclusão: a perda média é igual; só $1/r$ distingue as redes. Na segunda parte, a mesma ideia descreve os picos de chegada ao RU.

Intervalo

10 minutos

Segunda parte: dicas para o seminário da fila do RU

A equipe leva a fila do RU pelas cinco etapas de um estudo de desempenho:



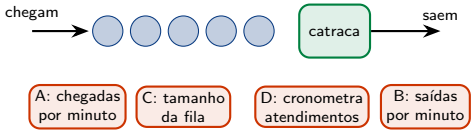
Antes de tudo

Escrevam a pergunta da equipe em uma frase, com um número. Exemplo: *um segundo atendente no pico reduz a espera para menos de 5 minutos?* A mesma pergunta guia as cinco etapas.

Aferir: como observar uma fila com muita gente

Não dá para anotar a entrada e a saída de cada aluno. A saída: **contar minuto a minuto** e cronometrar só uma **amostra**.

- **A**, no fim da fila: **chegadas** por minuto (contador no celular).
- **B**, na catraca: **saídas** por minuto.
- **C**: tamanho da **fila** no início de cada minuto (amostragem, Aula 06).
- **D**: cronometra 20 a 30 atendimentos.



Dicas: marcos fixos agilizam a contagem ("até a pilastra $\approx$ 15 alunos"); cubram o pico com folga; um só relógio; piloto de 10 min dias antes.

Aferir: a planilha por minuto e o que sai dela

Uma linha por minuto. Das três contagens sai todo o resto:

minuto	chegaram	saíram	no sistema
11:40	2	2	3
11:41	3	3	3
11:42	5	3	5
11:43	4	3	6
⋮	⋮	⋮	⋮

no sistema = fila + quem está na catraca

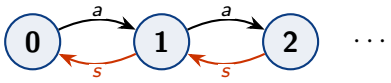
λ	média de "chegaram"
μ	média de "saíram" nos minutos **com fila**, $\div c$
$\rho = \lambda/(c\,\mu)$	se $\rho \geq 1$, a fila só cresce
L	média de "no sistema"
$W = L/\lambda$	tempo no sistema (lei de Little)

Duas conferências

(1) "no sistema" varia de chegaram − saíram: $5 + 4 - 3 = 6$; se não bater, alguém contou errado. (2) Os atendimentos cronometrados devem ter média perto de $60/\mu$ segundos.

Modelar: a fila como cadeia de Markov em quatro decisões

1. **Estado:** o que muda e importa. RU: n = alunos na fila + em atendimento.
2. **Passo Δt:** no máximo um evento por passo; bem menor que $60/\lambda$ e $60/\mu$ s.
3. **Probabilidades:** chegada $a = \lambda\,\Delta t$; saída $\min(n, c)\,s$, com $s = \mu\,\Delta t$; ficar = o resto.
4. **Capacidade K:** o maior n possível.



Exemplo ilustrativo

$\lambda = 2{,}4$/min e $\mu = 3$/min: chega um aluno a cada 25 s e um atendimento termina a cada 20 s. Com $\Delta t = 5$ s: $a = 2{,}4 \times \frac{5}{60} = 0{,}2$ e $s = 3 \times \frac{5}{60} = 0{,}25$.

Confira $a + c\,s \leq 1$ (senão, diminua Δt). As quatro decisões cabem em um slide do seminário.

Modelar: fases de calma e pico com os dados da equipe

1. **Chegadas por minuto** (planilha ilustrativa, média 2,4/min):

1	2	1	2	2	1	**4**	**5**	**4**	**4**	**4**	2	1	2	1
C	C	C	C	C	C	P	P	P	P	P	C	C	C	C

2. **Rotulem:** pico (P) com 4 ou mais chegadas, cerca de 1,5 vez a média.
3. **Contem os pares** C→P e P→C, como na estimação de P: $p \approx 1/9$ e $r = 1/5$ (modelo de Gilbert).
4. **Taxa de cada fase:** calma 1,5/min; pico 4,2/min. No pico, $\rho = 4{,}2/3 = 1{,}4 > 1$: a fila cresce.

No modelo: o estado passa a ser (fase, n); a cada passo a fase pode trocar (p ou r) e a chegada usa o a da fase.

Armadilha: uma hora tem poucos picos. Juntem dias diferentes ou declarem como ressalva.

Simular e validar

Simulador da equipe

Próprio, de tecnologia livre e **à escolha da equipe** (web, Python, desktop…). Deve mostrar a fila passo a passo com a fase, um almoço das 11h às 13h, as métricas e as configurações de atendentes.

Dicas

- comecem **sem fases**; acrescentem depois;
- semente fixa para achar erros;
- em 100 h, as métricas batem com π.

Validação

Um almoço é uma amostra só. Comparem a medição com a **faixa de 90%** de 500 almoços simulados.

Dicas

- usem uma medida que o modelo errado **não** imita: D = variância $\div$ média das chegadas por minuto (≈ 1 sem fases);
- se não bater: revejam as contagens, as fases e o CV dos atendimentos.

Recomendar e apresentar

Cenários "e se?"

- 2.º atendente só no pico (c);
- atendimento mais rápido (μ);
- saídas de aula escalonadas.

Um parâmetro por vez, com **ganho e custo**.

Conclusão que convence

"Das 11h40 às 12h10 chegam 4,1 alunos/min para 3 atendimentos/min ($\rho = 1{,}4$). Um 2.º atendente só nesse intervalo reduz a espera simulada de 9 para 2 min." (ilustrativo)

Diz **quando**, **quanto** e **por quê**. Evitem: "a fila é grande e precisa de mais atendentes".

Pergunta 2 min	Aferição 4 min	Modelo 5 min	Simulação e validação 5 min	Recomendação e limites, 4 min

Demonstração do simulador de até 2 min, com vídeo de reserva. Ensaiem com cronômetro.

Trabalho final: artigo de pesquisa sobre cadeias de Markov

Objetivo

Pesquisar uma aplicação real de cadeias de Markov em computação e apresentá-la, com a notação da disciplina, em um **artigo curto**, discutido em sala em 10/11.

Estrutura do artigo

1. Introdução: o sistema e a pergunta
2. A cadeia: estados, passo e P
3. O que se calcula com ela
4. Exemplo numérico em Python
5. Limites e validação
6. Conclusão e referências

Formato

- em dupla: um tema e um recorte;
- 4 a 6 páginas, modelo da SBC;
- ponto de partida e mais duas fontes acadêmicas;
- entrega em PDF no SIGAA;
- IA pode ajudar a revisar texto e código; toda referência deve ser **conferida** na fonte.

Três temas: do clássico à inteligência artificial

1. PageRank (1998)

Clássico, Web. Por que a π de um navegante aleatório ordena as páginas?

Recortes: amortecimento; páginas sem saída; PageRank fora da Web.

Partida: Brin e Page (1998).

2. Perdas em rajada

Clássico, redes. Mesma perda média, qualidade diferente?

Recortes: estimar p e r de um traço; Bernoulli $\times$ Gilbert; efeito em voz e vídeo.

Partida: Gilbert (1960); Yajnik et al. (1999).

3. Linguagem e IA

Moderno. Do n-grama ao ChatGPT: até onde vale Markov?

Recortes: bigramas em português; temperatura; LLM como cadeia.

Partida: Shannon (1948); Zekri et al. (2024).

Referências: `trabalho09-artigo.pdf` · até **2 duplas por recorte** · discussão em sala: 10/11

Avaliação (4 pontos): ficha da cadeia (0,8), exemplo em Python (0,8), limites e validação (0,8), texto e referências (0,8), participação (0,8).

Próxima aula

Aula 10 (03/11/2026): processos de nascimento e morte; a fila M/M/1

- A cadeia da fila do RU, com taxas no lugar de probabilidades.
- Uma chegada avança um estado (taxa λ); um atendimento recua um estado (taxa μ).
- Com passos cada vez menores, $a = \lambda\,\Delta t$ e $s = \mu\,\Delta t$ dão lugar às **taxas** λ e μ: o tempo passa a ser contínuo.
- O equilíbrio dessa cadeia fornece as métricas da fila por fórmula, conferidas com o simulador de cada equipe.

Referências

JAIN, R. *The Art of Computer Systems Performance Analysis*. New York: Wiley, 1991.

JOHNSON, T. de M. e S. M. *Avaliação de desempenho de sistemas computacionais*. TrioStudio, 2011.

TIJMS, H. C. *A First Course in Stochastic Models*. Chichester: Wiley, 2003. (seção 3.2: estados absorventes)

HARCHOL-BALTER, M. *Performance Modeling and Design of Computer Systems: Queueing Theory in Action*. Cambridge: Cambridge University Press, 2013. (parte IV: das cadeias de Markov às filas)

DADUNA, H. *Queueing Networks with Discrete Time Scale*. Berlin: Springer, 2001 (LNCS 2046).

GILBERT, E. N. Capacity of a burst-noise channel. *Bell System Technical Journal*, v. 39, n. 5, p. 1253–1265, 1960.

ELLIOTT, E. O. Estimates of error rates for codes on burst-noise channels. *Bell System Technical Journal*, v. 42, n. 5, p. 1977–1997, 1963.

ZUKERMAN, M. *Introduction to Queueing Theory and Stochastic Teletraffic Models*. arXiv:1307.2968, 2013. (fila com chegadas moduladas por Markov comparada à M/M/1 de mesma taxa média)

PyDTMC. Disponível em: `https://pydtmc.readthedocs.io`. Testado com a versão 10.1.